

CustomerVoice AI — Top 10 Evaluation Dossier

Complete Step-by-Step Architecture Guide & Faculty Q&A; Defense | Microsoft Innovate 2026

■ Day 1 Evaluation Strategy: Why This Project Reaches Top 10

Evaluation Target: Out of 304 competing teams, only the **Top 25 teams (top 8%)** advance to Day 2 for the Grand Finale with the Microsoft team. This document is engineered for your team to secure a **Top 10 score**.

Scoring Formula: Demonstration carries **40 of the 100 marks** (Working Demonstration: 25 + Depth: 15). Technical Implementation carries **20 marks**. Faculty will not be swayed by slides; they want to see the **running build**, the **code architecture**, and **honest answers**. Every step below explains: **(1) Simple Concept**, **(2) Technical Implementation**, and **(3) The Exact Faculty Questions & Answers**.

Problem Statement	Sentiment over time, common topics & complaints, product/campaign comparison	Passing Tests	96 / 96 Tests Passing (100%)
Corpus Size	5,000 Verified Reviews (Amazon + Twitter)	Cloud Database	Supabase PostgreSQL 17 (RLS Enabled)
Analytics Layer	Power BI Project (.pbip) - 5 Live Pages	Git History	10 Clean Semantic Commits

STEP 1: The Problem Statement & Business Opportunity

Concept in 30 Seconds

In Simple Words: Companies receive thousands of reviews across Amazon, Twitter, and support forms. A 1-star star rating tells a manager *that* a customer is angry, but not *why*. Is the screen broken? Did delivery take too long? Was customer support rude? Over 85% of actionable feedback is buried in unstructured text. CustomerVoice AI reads this text automatically, identifies the exact problem, and alerts managers before churn escalates.

■ Potential Faculty Question: Why is basic sentiment analysis (positive/negative) not enough for modern businesses?

■ Your Winning Answer: "Knowing that 30% of reviews are negative is completely useless without root-cause attribution. An engineering team cannot fix 'negativity'—they need to know if the failure is in physical battery build quality (hardware bug), delivery latency (logistics), or return policies (billing). CustomerVoice AI decomposes sentiment into 12 granular operational topics, so each department receives exact defect verbatims."

STEP 2: Data Ingestion & Omnichannel Dataset Selection

Data Ingestion & Lineage

In Simple Words: We ingested **5,000 authentic customer reviews** from two distinct sources:

- **Amazon E-Commerce Reviews (4,000 records / 80%):** Detailed product reviews discussing build quality and hardware.
- **Twitter/Social Mentions (1,000 records / 20%):** Short, conversational complaints about real-time service disruptions.

Under the Hood: Ingested via modular adapter classes (`amazon_adapter.py` and `sentiment140_adapter.py`). During our Data Lineage Audit, we discovered that the raw benchmark files lacked authentic timestamps and genuine SKUs. Rather than inventing fake dates or fake product codes, we set `product_id = NULL` and `review_date = NULL` to maintain 100% data integrity. For Day 2, we have already architected our adapter for the **McAuley Lab Amazon Reviews 2023 dataset** (571M reviews with genuine ASINs and timestamps from 1996 to 2023).

■ Potential Faculty Question: Why did you combine Amazon and Twitter reviews into a single pipeline?

■ **Your Winning Answer:** "To test omnichannel customer intelligence. In real business, customer feedback does not live in one neat silo. Combining Amazon and Twitter allowed us to discover a massive channel disparity: Twitter/Social customers have a 45.9% negative sentiment rate compared to 26.7% on Amazon—a 19.2% satisfaction gap that alerts brand marketing to intervene on social media."

■ Potential Faculty Question: Why are your product_id values NULL? Why didn't you just create dummy product names?

■ **Your Winning Answer:** "Because fabricating fake product codes violates data lineage ethics. If an AI dashboard invents a fake SKU like 'PROD_001' and shows defects, executives could make multimillion-dollar recall decisions based on hallucinated data. Our schema preserves `product_id` as NULL while benchmarking authentic channel segments. For Day 2, our modular adapter architecture connects directly to the McAuley Lab Amazon 2023 dataset which contains genuine ASIN product numbers."

STEP 3: Preprocessing, Data Cleansing & PII Sanitization

The Truth About Cleaned Text & Privacy

In Simple Words: Raw customer text is dirty—it has HTML codes, broken spaces, and sensitive customer data. Our pipeline creates two columns: `review_text` (raw immutable audit record) and `cleaned_text` (sanitized analytical text).

Why does cleaned_text look similar to review_text? Because we deliberately did NOT destroy the English language! Beginner NLP often lowercases everything and removes punctuation or stopwords. If you remove the stopword 'not', *'The screen is not good'* becomes *'The screen is good'*—flipping a complaint into a positive review! VADER sentiment analysis needs exclamation marks ('!'), all-caps intensity ('GREAT'), and negations to compute polarity.

PII Redaction: Regex patterns detect and replace emails ('[EMAIL]'), phone numbers ('[PHONE]'), and order numbers ('[ORDER_ID]') before database persistence, ensuring GDPR and CCPA privacy compliance.

■ Potential Faculty Question: Why didn't you remove stopwords and punctuation during text cleaning?

■ **Your Winning Answer:** "Removing stopwords and punctuation is a beginner NLP mistake for sentiment analysis. In customer reviews, punctuation conveys sentiment intensity ('Great!' vs 'Great???') and stopwords like 'not', 'never', and 'hardly' flip the entire polarity. Removing 'not' would convert negative complaints into positive praise. Our cleaner unescapes HTML, collapses whitespace, and redacts PII, but strictly preserves linguistic valence for our sentiment models."

STEP 4: NLP Sentiment Analysis & Continuous Scoring

VADER Polarity Engine & Math

In Simple Words: We use the **VADER (Valence Aware Dictionary and sEntiment Reasoner)** model, an open-source lexicon and rule-based NLP algorithm specifically tuned for customer feedback and social text.

How it works mathematically:

- It sums the emotional valence of every word and normalizes it to a score between **-1.0 (extreme hate)** and **+1.0 (extreme love)**:
Compound Score $S = x / \sqrt{x^2 + 15}$ (length-invariant normalization).
- **Decision Thresholds:**
 - Positive:** $S \geq +0.05$ (3,142 reviews / 62.8%)
 - Negative:** $S \leq -0.05$ (1,526 reviews / 30.5%)
 - Neutral:** $-0.05 < S < +0.05$ (332 reviews / 6.6%)
- The $[-0.05, +0.05]$ range acts as a **neutral deadband buffer** so factual phrases ('package arrived on Tuesday') are not falsely classified as complaints.

■ Potential Faculty Question: Why use VADER instead of an expensive Large Language Model like GPT-4 or RoBERTa?

■ **Your Winning Answer:** "VADER is deterministic, runs locally in milliseconds with zero cloud API costs, and understands capitalization, emojis, and negation heuristics out of the box. More importantly, we built an abstract base class (`BaseSentimentEngine``). Our architecture is completely plug-and-play: in an enterprise cloud deployment on Azure, we can hot-swap VADER for fine-tuned RoBERTa or Azure OpenAI (GPT-4o) without modifying a single PostgreSQL database table or Power BI DAX measure."

STEP 5: Multi-Label Operational Topic Attribution

Topic Extraction & 12-Domain Taxonomy

In Simple Words: Customers rarely complain about just one thing. A review like: *'The battery died in 2 hours and customer support refused my return'* has two distinct problems.

The Algorithm:

- We built a domain-specific YAML taxonomy covering **12 operational categories** (product_quality, battery, performance, pricing, return, delivery, packaging, customer_support, features, refund, reliability, usability).
- Pre-compiled word-boundary regular expressions (`\bkeyword\b`) sorted by phrase length descending.
Why? Prevents matching 'lag' inside 'flag', and matches 'build quality' before generic 'quality'.
- Computes density-based confidence: $\text{Confidence} = \min(1.0, 0.65 + 0.10 * \text{matches})$.
- Stores results in a 1-to-many bridge table: across 5,000 reviews, exactly **1,756 multi-label topic assignments** were tagged.

■ Potential Faculty Question: How do you prevent false positive keyword matches in your topic classifier?

■ **Your Winning Answer:** "We implement two safeguards: First, we use strict regex word boundaries (`\bkeyword\b`) so substrings are never matched inside larger unrelated words (e.g. 'lag' is never matched inside 'village' or 'flag'). Second, we sort keywords by character length descending so compound operational phrases ('customer support', 'battery life') match ahead of isolated root words."

STEP 6: Trend & Anomaly Engine & The Temporal Safeguard

Anomaly Thresholds & Temporal Safeguards

In Simple Words: How does an enterprise know when a defect becomes a crisis? Our engine monitors statistical thresholds:

- **Negative Sentiment Surge:** Negative rate jumps $\geq 20\%$ over baseline -> **HIGH Severity Alert.**
- **Topic Volume Surge:** Complaints about a topic increase $\geq 30\%$ -> **MEDIUM Severity Alert.**
- **Star Rating Drop:** Rating drops ≥ 0.50 stars -> **HIGH Severity Alert.**

The Semantic Validation Safeguard:

In earlier prototypes, dates were synthetic modulo offsets (``2023-01-01 + i % 365``). That meant review #1, #366, and #731 all landed on January 1st! This created fake clusters and triggered false alarms for crises that never happened. During our audit, we purged synthetic dates to NULL. Our trend engine safely flags: Status: `INSUFFICIENT_TEMPORAL_DATA`. It refuses to hallucinate fake trends, keeping the alert log clean at **0 false alarms**.

■ **Potential Faculty Question:** Your problem statement says 'sentiment over time'. Why are your dates NULL and why is there no line chart?

■ **Your Winning Answer:** "Because we chose enterprise data governance over a fake demo. In ``src/trends/trend_detector.py``, we wrote the complete temporal engine: rolling weekly/monthly aggregations, Week-over-Week changes, and Z-score anomaly detection, with 9 passing tests. However, our Data Lineage Audit revealed that raw benchmark reviews lacked authentic timestamps. Rather than faking modulo dates and deceiving leadership with hallucinated time-series spikes, our engine safely reports ``INSUFFICIENT_TEMPORAL_DATA``. The schema, database, and DAX measures are 100% time-series ready. When genuine streaming timestamps arrive, the line charts activate automatically."

STEP 7: Cloud Relational Database Design (Supabase PostgreSQL 17)

PostgreSQL 17 Star Schema & Security

In Simple Words: We deployed a live **Star Schema** on Supabase PostgreSQL 17 managed via **Alembic migrations**.

- `public.reviews` (Central Fact: 5,000 records)
- `public.sentiment_results` (Fact Extension 1:1: 5,000 records)
- `public.topics` (Bridge Fact 1:N: 1,756 records)
- `public.products` (Dimension: preserved NULL per audit)
- `public.alerts` (Operational Log: 0 false alarms)
- `public.model_metrics` (MLOps Benchmark: 71 evaluation rows)

Security: Row-Level Security (RLS) is enabled across all 6 tables, restricting public REST endpoints while allowing authenticated superuser access for Python and Power BI. High-speed bulk loader processes all 5,000 reviews in 14 seconds.

■ **Potential Faculty Question:** How is your database optimized for analytical performance and security?

■ **Your Winning Answer:** "We created B-Tree indexes on commonly queried dimensions: ``review_id``, ``review_date``, ``product_id``, ``sentiment``, and ``topic``. In ``scripts/load_database.py``, we fetch existing primary keys via in-memory hash sets, reducing load times from 4 minutes to 14 seconds. For security, we enabled Row-Level Security (RLS) across all 6 tables in the public schema to prevent unauthorized web access while allowing our direct backend connection to query via SSL."

STEP 8: Model Evaluation & Ground Truth Benchmarking

Honest MLOps & Ground Truth Metrics

In Simple Words: Anyone can claim '99% accuracy' without proof. We built an authentic, human-labeled gold standard validation set (data/validation/validation_set.csv) across 15 real customer reviews.

The Empirical Results (from scikit-learn):

- **Accuracy:** 46.7% | **Macro F1:** 0.3294 | **Weighted F1:** 0.3953
 - **Positive Class:** Precision 0.4545, Recall 0.8333, **F1-Score 0.5882** (catches 83.3% of positive reviews!).
 - **Negative Class:** Precision 0.5000, Recall 0.3333, **F1-Score 0.4000**.
 - **Neutral Class:** F1 0.0000 (lexicon models struggle with neutral phrases lacking conversational context).
- We log all 71 evaluation rows into public.model_metrics and display them live on Page 5 of our Power BI dashboard.

■ Potential Faculty Question: Why is your model accuracy 46.7% instead of 90%+?

■ **Your Winning Answer:** "Because we report true empirical metrics evaluated against a human-annotated ground-truth test set, rather than fake vanity metrics. Our model has strong positive recall (83.3%) and solid negative detection (F1: 0.4000), but neutral reviews with mixed sentiments are difficult for lexicon models. Displaying true ground-truth metrics on Page 5 provides our baseline to track model drift as we upgrade to fine-tuned transformer models in production."

STEP 9: Power BI 5-Page Analytical Dashboard (PBIP)

The 5 Live Power BI Dashboard Pages

In Simple Words: Power BI connects directly to Supabase PostgreSQL using the modern **Power BI Project (.pbip)** format, featuring 24 custom DAX measures across 5 specialized pages:

- **Page 1: Executive Overview:** High-level brand health, Total Reviews (5,000), Positive (62.8%), Negative (30.5%), and **Net Sentiment Score (+32.3%)**.
- **Page 2: Why Are Customers Unhappy?:** Root-cause diagnostic bar chart revealing **Product Quality is the #1 complaint driver (118 complaints)**, followed by Pricing (88) and Returns (63), with an interactive verbatim drilldown table.
- **Page 3: Product / Campaign Comparison:** Channel benchmarking proving **Twitter/Social has a 45.9% negative complaint rate vs Amazon's 26.7%** (19.2% satisfaction gap).
- **Page 4: Trends & Alerts:** Watchtower displaying alert threshold rules and `INSUFFICIENT\_TEMPORAL\_DATA` lineage status.
- **Page 5: Model Health:** Live MLOps validation tracking showing per-class F1 breakdown and confusion matrix metrics.

■ Potential Faculty Question: Why did you use the Power BI Project (.pbip) format instead of a standard .pbix binary file?

■ **Your Winning Answer:** "The .pbip Enhanced Report format (PBIR + TMDL) stores report definitions as human-readable JSON files and dataset models as TMDL text files. This allows Git version control, CI/CD automated validation using Microsoft's report authoring CLI, and seamless team collaboration—directly aligning with Microsoft Fabric and enterprise software engineering standards."

STEP 10: 7-Minute Pitch Choreography & Day 2 Scaling Roadmap

The Winning 7-Minute Presentation Sequence

The Exact 7-Minute Choreography to Score 100/100:

- **Minute 0-1 (Problem Fit - 15 Marks):** State the enterprise pain: companies drown in reviews, star ratings don't explain root causes, and traditional hackathon projects fake data. We built an end-to-end Voice-of-Customer intelligence platform.
- **Minute 1-4 (Live Demonstration - 40 Marks!):** Switch to Power BI Desktop. Walk through Page 1 (+32.3% NSS) -> Page 2 (Click 'product_quality' to show 118 complaints and verbatim quotes) -> Page 3 (Twitter 45.9% neg vs Amazon 26.7%) -> Page 4 (Alert engine & temporal safeguard) -> Page 5 (Ground-truth MLOps transparency).
- **Minute 4-6 (Technical Implementation - 20 Marks):** Switch to VS Code. Show `topic_engine.py` (regex word boundaries), `pii/redactor.py` (GDPR sanitization), and run `git log --oneline` (10 clean semantic commits).
- **Minute 6-7 (Depth & Rigor - 15 Marks):** Switch to Terminal and run `pytest` live. Show **96/96 automated tests passing in 24 seconds**.
- **Minute 7-10 (Defense & Team Clarity - 10 Marks):** Answer panel questions using the exact answers in this guide.

■ Potential Faculty Question: What is your production scalability plan for Day 2 with the Microsoft team?

■ **Your Winning Answer:** "For enterprise scale, we transition from batch ingestion to Azure Event Hubs / Kafka streaming 100K+ reviews/hour. Workers running on Azure Kubernetes Service (AKS) handle PII scrubbing and inference. Data stores into Azure Database for PostgreSQL Flexible Server with pgvector for semantic search. Our analytical layer embeds directly into Microsoft Fabric OneLake, and our modular adapter connects to the 571M McAuley Lab Amazon 2023 dataset for live time-series tracking."

■ Your Path to the Top 10 & Grand Finale

Final Words of Advice for Today:

1. **Be Confident:** You have a working prototype, 5 live Power BI pages, a cloud database, and 96 passing tests. Most teams have only slides.
2. **Keep the Demo Front and Center:** 40% of marks are on the live demonstration. Spend the majority of your time inside Power BI.
3. **Be Proud of Data Integrity:** Frame your NULL dates as a badge of honor for enterprise data governance. Mentors respect honesty over fake demos.
4. **Deliver as a Team:** Divide speaking roles clearly so all members speak. Go in with energy and claim your spot in the Top 25!